

Stroke Prediction Analytics

Sprint 1: Data Cleaning | Sprint 2: EDA & Visualization | Sprint 3: Statistics & Feature Engineering

Tools used: Python · Pandas · NumPy · Matplotlib · Seaborn · SciPy

Dataset: https://drive.google.com/file/d/1Z_Z67Fv5JjOpTWjxNH86CYtfWupuUR_v/view?usp=sharing

Sprint 1 – Data Understanding & Preparation

1. Import Core Libraries

Which libraries do we need before touching the data?

```
import pandas as pd
import numpy as np
```

Why these variables: pandas (aliased pd) is the standard library for loading and manipulating tabular data as a DataFrame. numpy (aliased np) is needed later for select_dtypes(include=np.number) to separate numeric from text columns. These are the two minimum libraries any tabular analysis needs.

Key Insight: No output yet — this cell just prepares the toolkit for everything that follows.

2. Load and Preview the Raw Dataset

How do I load the CSV and take a first look at it?

```
df = pd.read_csv("/content/healthcare-dataset-stroke-data.csv")
df
df.head()
df.tail()
```

Why these variables: The variable name df (short for DataFrame) is the standard convention so every later line of code (df['bmi'], df.shape, etc.) stays short and readable. head() and tail() were used instead of printing the whole table because with 5,110 rows, a full print would be unreadable — checking the first and last rows is enough to confirm the file loaded with the right columns.

Key Insight: The dataset loaded successfully with columns like id, gender, age, hypertension, heart_disease, ever_married, work_type, Residence_type, avg_glucose_level, bmi, smoking_status, and stroke. The tail already shows a NaN pattern in bmi, an early hint that missing values need handling.

3. Rows, Columns, and Feature List

How many rows/columns are there, and what are the column names?

```
df.shape[0] #Number of rows
df.shape[1] #Number of columns
df.columns #Features available
```

Why these variables: shape[0] and shape[1] were used (rather than len(df) and len(df.columns)) because shape returns both numbers in one call and is the pandas-standard way to check table size. df.columns lists every feature so we know exactly what's available before deciding which are useful.

Key Insight: The dataset has 5,110 rows and 12 columns: id, gender, age, hypertension, heart_disease, ever_married, work_type, Residence_type, avg_glucose_level, bmi, smoking_status, stroke.

4. Identify the Target Variable

Which column are we actually trying to explain/predict?

```
target_variable = "stroke"
```

Why these variables: "stroke" was chosen as the target because it is the binary outcome (1 = had a stroke, 0 = did not) that the hospital management specifically wants explained — every other column is a potential predictor of it, not the outcome itself.

Key Insight: stroke is confirmed as the target variable for the entire project — every analysis from here on is measured against how it relates to stroke.

5. Separate Numeric vs Categorical Columns

Which columns are numbers, and which are text/category labels?

```
df.select_dtypes(include=np.number).columns #Numerical columns
categorical_columns = df.select_dtypes(object).columns #Categorical columns
categorical_columns
```

Why these variables: `select_dtypes(include=np.number)` and `select_dtypes(object)` were used instead of manually typing out each column name, because it automatically and reliably sorts columns by their actual stored data type — this avoids typos and stays correct even if columns are added later. `categorical_columns` is saved into a variable because it gets reused later (in the cleaning-loop step) instead of being retyped.

Key Insight: Numeric columns: id, age, hypertension, heart_disease, avg_glucose_level, bmi, stroke. Categorical columns: gender, ever_married, work_type, Residence_type, smoking_status. This split is what later decides which charts (histograms vs. countplots) and which statistical tests (T-test vs. Chi-square) apply to each column.

6. DataFrame Exploration — Shape, Types, Summary

How do I get a full structural check of the dataset in one go?

```
df.shape #Shape analysis
df.dtypes #Data types
df.info() #Column information
df.describe() #Summary statistics
```

Why these variables: All four were run together because each answers a different structural question with almost no extra effort: `shape` confirms size, `dtypes` flags any column stored as the wrong type, `info()` adds non-null counts (revealing missing data at a glance), and `describe()` gives min/max/mean for every numeric column so we can immediately spot impossible values (like a negative age).

Key Insight: `df.info()` revealed bmi has only 4,909 non-null values out of 5,110 — meaning 201 rows are missing bmi. `df.describe()` showed age ranges realistically from 0.08 to 82, but `avg_glucose_level` goes up to 271.74 and bmi up to 97.6 — both worth checking as potential outliers later.

7. Find Missing Values

Exactly how many missing values are in each column?

```
df.isnull().sum() # missing values
```

Why these variables: `isnull().sum()` was used instead of just `isnull()` alone because `isnull()` only returns True/False per cell — adding `.sum()` turns that into one clear count per column, which is what we actually need to decide where cleaning is required.

Key Insight: Only bmi has missing values — exactly 201 out of 5,110 rows (about 4%). Every other column (gender, age, hypertension, etc.) has zero missing values, so cleaning effort only needs to focus on bmi.

8. Fill Missing BMI Values

How should the 201 missing BMI values be filled in?

```
median_bmi = df['bmi'].median() #handling missing values
df['bmi'] = df['bmi'].fillna(median_bmi)
null = df['bmi'].isnull().sum()
print("Missing values in bmi after median imputation:", null)
```

Why these variables: median_bmi (not mean_bmi) was chosen deliberately: BMI has extreme high outliers (up to 97.6), and the mean would be dragged upward by them, giving a falsely high 'typical' BMI. The median ignores that distortion and represents the true middle patient. null is just a quick re-check variable to confirm the fix worked, not a new metric.

Key Insight: After filling, printed output confirmed: "Missing values in bmi after median imputation: 0" — all 201 missing BMI values are now filled with the sample median, and the column is complete.

9. Check and Remove Duplicate Rows

Are any patient records repeated in the dataset?

```
df.duplicated().sum() #duplicate values
df = df.drop_duplicates() #optional because there are no duplicate values
```

Why these variables: duplicated().sum() is the standard one-line way to count exact repeated rows. drop_duplicates() was still run (even though the count was 0) to make the cleaning pipeline complete and safe to reuse if the raw data ever changes and does contain duplicates in future.

Key Insight: The count returned 0 — there are no duplicate patient records in this dataset, so no rows were actually removed by the drop_duplicates() call.

10. Convert Flag Columns to Category Type

How do I make sure stroke, hypertension, and heart_disease are treated as labels, not numbers?

```
df["stroke"] = df["stroke"].astype("category")
df["hypertension"] = df["hypertension"].astype("category")
df["heart_disease"] = df["heart_disease"].astype("category") # handle data types
```

Why these variables: These three specific columns were converted (and not, say, age or bmi) because they are stored as 0/1 integers but are really Yes/No labels — averaging or adding them like real numbers would be meaningless. Converting to "category" type tells pandas and later plotting functions to treat them as groups, not quantities.

Key Insight: No numeric output here — but this correctly prepares stroke, hypertension, and heart_disease to be used as grouping variables (x-axis / hue) in the countplots and crosstabs that come later in Sprint 2.

11. Check for Invalid / Impossible Values

Could there be negative ages, zero BMI, or zero glucose readings that signal bad data entry?

```
df[df["age"]<0] #Invalid values
df[df["bmi"]<=0]#Invalid values
df[df["avg_glucose_level"]<=0] #Invalid values
```

Why these variables: age<0, bmi<=0, and avg_glucose_level<=0 were chosen as the specific checks because these are the three numeric health columns where a value at or below zero is medically impossible — this is the fastest way to catch data-entry errors in exactly the columns where they'd matter most.

Key Insight: All three checks returned an Empty DataFrame — meaning there are no negative ages and no zero/negative BMI or glucose readings anywhere in the dataset. The numeric columns pass this validity check cleanly.

12. Check Categorical Columns for Typos/Odd Categories

Do the text columns (gender, smoking status, etc.) contain any unexpected or misspelled values?

```
print(df["stroke"].unique())
print(df["hypertension"].unique())
print(df["heart_disease"].unique()) #Invalid values
print(df["gender"].unique()) #Invalid values
print(df["smoking_status"].unique()) #Invalid values
print(df["work_type"].unique()) #Invalid values
print(df["Residence_type"].unique()) #Invalid values
```

Why these variables: .unique() was run individually on every categorical column (rather than just eyeballing a sample of rows) because it's the only way to guarantee every distinct value in the whole column — including a rare typo that shows up in only 1 row — gets caught.

Key Insight: gender includes an 'Other' category with very few records; smoking_status includes an 'Unknown' category (a large chunk of missing information, not a typo); work_type includes 'children' and 'Never_worked'. No genuine typos were found, but the casing was inconsistent (e.g. lowercase 'urban' vs 'Urban'), which is fixed in the next step.

13. Standardize Text Categories

How do I fix inconsistent spacing/capitalization across all categorical columns at once?

```
for col in categorical_columns:
    df[col] = df[col].str.strip().str.title()
for col in categorical_columns:
    print(col)
    print(df[col].unique()) #Inconsistent categorical values
```

Why these variables: A for loop over categorical_columns (the variable saved back in Task 5) was used instead of repeating .str.strip().str.title() five separate times — this guarantees every categorical column gets the exact same cleaning rule with no column accidentally skipped.

Key Insight: After cleaning, all categories now follow consistent Title Case, e.g. work_type shows 'Private', 'Self-Employed', 'Govt_Job', 'Children', 'Never_Worked' and smoking_status shows 'Formerly Smoked', 'Never Smoked', 'Smokes', 'Unknown' — no more mismatched casing that could split one real category into two.

14. Visualize Outliers with Boxplots

Do age, glucose level, and BMI show any visibly extreme values?

```
import seaborn as sns
import matplotlib.pyplot as plt
sns.boxplot(y=df['age'])
plt.title('Box Plot of Age'); plt.ylabel('Age'); plt.show()

sns.boxplot(y=df['avg_glucose_level'])
plt.title('Box Plot of Average Glucose Level'); plt.ylabel('Average Glucose Level');
plt.show()

sns.boxplot(y=df['bmi'])
plt.title('Box Plot of BMI'); plt.ylabel('BMI'); plt.show()
```

Why these variables: These three specific numeric columns (age, avg_glucose_level, bmi) were chosen for boxplots because they are the only continuous, non-flag numeric variables in the dataset — id is just a row identifier, and hypertension/heart_disease/stroke are 0/1 flags that a boxplot can't meaningfully summarize.

Key Insight: The age boxplot showed no visible outlier dots — ages spread naturally across the box. The glucose and BMI boxplots both showed a cluster of dots above the upper whisker, visually confirming that these two columns contain outliers worth investigating with the exact IQR calculation next.

15. Count Outliers Using the IQR Rule

Exactly how many outlier values are in age, BMI, and glucose level?

```
# count of outliers
columns = ["age", "bmi", "avg_glucose_level"]
for col in columns:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outliers = df[(df[col] < lower) | (df[col] > upper)]
    print(col)
    print("Outliers :", len(outliers))
```

Why these variables: Q1 (25th percentile) and Q3 (75th percentile) were used because together they define the 'normal' middle 50% of the data for each column. The 1.5×IQR rule is the standard statistical convention for flagging outliers — it was applied identically to all three columns inside one loop instead of writing the same five lines three separate times.

Key Insight: Results: age had 0 outliers, bmi had 126 outliers, and avg_glucose_level had 627 outliers. This confirms what the boxplots showed — age is well-behaved, while BMI and especially glucose level have a meaningful number of unusually high readings.

16. Calculate Exact BMI and Glucose Outlier Boundaries

What are the precise upper/lower cutoffs for BMI and glucose outliers?

```
#bmi
Q1 = df["bmi"].quantile(0.25)
Q3 = df["bmi"].quantile(0.75)
IQR = Q3 - Q1
lower_bmi = Q1 - 1.5 * IQR
upper_bmi = Q3 + 1.5 * IQR
print("Lower Limit :", lower_bmi)
print("Upper Limit :", upper_bmi)

# Average Glucose Level
Q1 = df["avg_glucose_level"].quantile(0.25)
Q3 = df["avg_glucose_level"].quantile(0.75)
IQR = Q3 - Q1
lower_glucose = Q1 - 1.5 * IQR
upper_glucose = Q3 + 1.5 * IQR
print("Lower Limit :", lower_glucose)
print("Upper Limit :", upper_glucose)
```

Why these variables: Separate lower_bmi/upper_bmi and lower_glucose/upper_glucose variables (rather than reusing the same lower/upper names) were kept, because both boundary pairs are needed again later to actually display and decide on the specific outlier rows — reusing one variable name would silently overwrite the first column's boundaries.

Key Insight: BMI outlier boundaries: below 10.30 or above 46.30. Glucose outlier boundaries: below 21.98 or above 169.36. Any BMI over ~46 or glucose over ~169 is statistically unusual and flagged for review.

17. Display and Review the Actual Outlier Rows

Which specific patients have these outlier BMI/glucose values, and what's the highest recorded?

```
bmi_outliers = df[(df["bmi"] < lower_bmi) | (df["bmi"] > upper_bmi)]
print(bmi_outliers)

glucose_outliers = df[(df["avg_glucose_level"] < lower_glucose) |
(df["avg_glucose_level"] > upper_glucose)]
print(glucose_outliers)

df["bmi"].sort_values(ascending=False).head(20)
df["avg_glucose_level"].sort_values(ascending=False).head(20)
```

Why these variables: Filtering with the lower_bmi/upper_bmi and lower_glucose/upper_glucose boundaries from Task 16 (instead of arbitrary round numbers) keeps the outlier definition mathematically consistent with the IQR rule. sort_values(ascending=False).head(20) was added to see the actual highest real-world values, not just the count, before deciding whether to keep them.

Key Insight: The highest BMI recorded is 97.6, and the highest average glucose level is 271.74. These are extreme but medically plausible (severe obesity, very high glucose) rather than data-entry errors — an important observation that supports the decision to keep them.

18. Decide: Keep or Remove the Outliers

Should these extreme BMI and glucose values be removed from the dataset?

```
#Decision
""" I want to retain the values because high outlier values in glucose can point to a
serious health problem and it may contribute to the stroke"""

#optional (NOT executed):
"""
df = df[(df["bmi"] >= lower_bmi) & (df["bmi"] <= upper_bmi)]
df = df[(df["avg_glucose_level"] >= lower_glucose) & (df["avg_glucose_level"] <=
upper_glucose)]
"""
```

Why these variables: The removal code was deliberately written inside a triple-quoted string (so it does NOT run) rather than as active code, because the decision was to retain the outliers — writing it as a comment/string still documents the alternative that was considered, without silently discarding real patient data that may be clinically meaningful for stroke prediction.

Key Insight: Decision: retain all outliers. Reasoning documented in the notebook: an extreme glucose reading can itself be a warning sign of diabetes and stroke risk, so removing it could hide exactly the pattern the whole project is trying to find.

19. Save the Cleaned Dataset

How do I save this cleaned version so Sprint 2 and 3 can reuse it?

```
df.to_csv("cleaned_healthcare_dataset.csv", index=False)
print("Dataset saved successfully!")
```

Why these variables: index=False was used specifically to stop pandas from writing an extra unnamed row-number column into the CSV — without it, every future re-load of this file would gain a useless extra column.

Key Insight: Printed confirmation: "Dataset saved successfully!" — cleaned_healthcare_dataset.csv now exists and is what Sprint 2 loads as the variable ok.

20. Final Data Validation

One last check — is the cleaned file really free of missing values, duplicates, and inconsistent columns?

```
df.isnull().sum()
df.duplicated().sum()
df.columns[df.columns.duplicated()]
for col in df.select_dtypes(include="object"):
    print(col)
    print(df[col].unique())
```

Why these variables: df.columns[df.columns.duplicated()] specifically checks for duplicate COLUMN NAMES (not duplicate rows) — a different failure mode that's easy to introduce accidentally (e.g. by merging data twice) and worth checking separately before calling the dataset final.

Key Insight: isnull().sum() returned 0 for every column, duplicated().sum() returned 0, and no duplicate column names were found. The categorical unique() re-check confirmed clean, consistent category labels. The dataset is officially validated and ready for analysis.

Sprint 1 Summary: 5,110 rows × 12 columns, target = stroke. bmi had 201 missing values, fixed with the median. No duplicate rows. 126 BMI outliers and 627 glucose outliers were found and deliberately retained for clinical relevance. Categorical text was cleaned to consistent Title Case. The result is cleaned_healthcare_dataset.csv, validated and ready for Sprint 2.

Sprint 2 – Exploratory Data Analysis & Visualization

1. Load the Cleaned Dataset

How do I bring in the cleaned file for this sprint?

```
ok = pd.read_csv("/content/cleaned_healthcare_dataset.csv")
ok
```

Why these variables: A new variable name, `ok`, (instead of reusing `df`) was used deliberately to clearly mark that Sprint 2 works on the cleaned output of Sprint 1, not the original raw file — this avoids ever accidentally re-analyzing dirty data.

 **Key Insight:** The cleaned dataset loads correctly with all 5,110 rows and the previously cleaned column values (Title Case categories, filled BMI).

2. Descriptive Statistics — Mean, Median, Mode

What are the central values of the important numeric columns?

```
numerical_columns = ["id", "age", "hypertension", "heart_disease", "avg_glucose_level",
"bmi", "stroke"]
mean = ok[numerical_columns].mean()
median = ok[numerical_columns].median()
mode = ok[numerical_columns].mode()
print(mean); print(median); print(mode)
```

Why these variables: `numerical_columns` was defined as its own list (rather than using `ok.mean()` on the whole table) so that `id` doesn't get accidentally treated as a meaningful measurement, and so this exact same column list can be reused for `describe()` right after.

Key Insight: Age: mean 43.23, median 45.00. Average Glucose Level: mean 106.15, median 91.88 — the mean is noticeably higher than the median, confirming glucose is right-skewed by high outliers. BMI: mean 28.86, median 28.10 — much closer together, meaning BMI is more evenly distributed.

3. Dispersion — Variance, Std Dev, Min/Max/Quantiles

How spread out are age, glucose, and BMI, and what shape does each distribution have?

```
print(ok.var(numeric_only=True))
print(ok.std(numeric_only=True))
print(ok.min(numeric_only=True))
print(ok.max(numeric_only=True))
print(ok.select_dtypes(include='number').quantile([0.25, 0.5, 0.75]))
print(ok.skew(numeric_only=True))
print(ok.kurtosis(numeric_only=True))
```

Why these variables: `numeric_only=True` was passed to every function so pandas automatically skips text columns like gender instead of raising an error — this was preferred over manually listing numeric columns again since we just want a full statistical sweep here.

Key Insight: `avg_glucose_level` has by far the largest variance (2,050.6) and standard deviation (45.28) of the three, confirming it's the most spread-out (and most skewed) numeric feature — consistent with the mean-vs-median gap found in the previous step.

4. Univariate Analysis — Histograms for Numeric Columns

What does the individual distribution of each numeric column look like?


```
num_cols = ["id", "age", "hypertension", "heart_disease", "avg_glucose_level",
            "bmi", "stroke"]
for col in num_cols:
    sns.histplot(ok[col], kde=True)
    plt.title(col)
    plt.show()
```

Why these variables: The same num_cols list from the descriptive-stats step was reused (rather than a new list) so every column that was measured numerically also gets its shape visualized — keeping the numeric analysis consistent end to end. A for loop was used instead of 7 separate plotting cells to avoid repeating identical code.

Key Insight: age looks close to a natural, broad spread across adult ages. avg_glucose_level is clearly right-skewed with a long tail toward high values (matching the earlier skew finding). hypertension, heart_disease, and stroke show tall bars at 0 and short bars at 1 — visual proof of severe class imbalance in these flags.

5. Univariate Analysis — Boxplots and Violin Plots

Can I see the outliers and full shape of the numeric columns together?

```
selected_cols = ['id', 'age', 'avg_glucose_level', 'bmi']
for col in selected_cols:
    sns.boxplot(y=ok[col])
    plt.title(col)
    plt.show()

for col in num_cols:
    sns.violinplot(y=ok[col])
    plt.show()
```

Why these variables: selected_cols drops id from the violin-plot list because id is just a row counter with no real distribution to interpret — including it would waste a chart. Violin plots were added on top of boxplots because a violin also shows the density/shape of the data, not just the five-number summary a boxplot gives.

Key Insight: The violin plots confirm the histogram finding visually: avg_glucose_level and bmi have a 'long tail' shape stretching upward, while age has a more balanced, rounder violin shape.

6. Univariate Analysis — Categorical Value Counts

How many patients fall into each category for gender, marital status, work type, etc.?

```
cat_cols = ['gender', 'ever_married', 'work_type', 'Residence_type', 'smoking_status']
for col in cat_cols:
    print(ok[col].value_counts())
    print("-"*40)

for col in cat_cols:
    sns.countplot(data=ok, x=col)
    plt.show()
```

Why these variables: cat_cols lists exactly the 5 text columns identified back in Sprint 1 Task 5 — reusing that same definition instead of retyping ensures the categorical analysis in Sprint 2 covers exactly the same columns that were cleaned in Sprint 1.

Key Insight: gender: 2,994 Female vs 2,115 Male vs only 1 Other. work_type: Private dominates with 2,925 patients. smoking_status: 1,892 Never Smoked but a large chunk (1,544) are Unknown — a real data-quality gap worth flagging to the hospital, since it limits how confidently smoking can be linked to stroke.

7. Bivariate Analysis — Numeric vs Numeric

Do age, glucose level, and BMI move together, and where do stroke cases sit?

```
sns.scatterplot(data=ok, x="age", y="avg_glucose_level", hue='stroke'); plt.show()
sns.scatterplot(data=ok, x="age", y="bmi", hue='stroke'); plt.show()
sns.scatterplot(data=ok, x="bmi", y="avg_glucose_level", hue='stroke'); plt.show()
```

Why these variables: hue='stroke' was added to every scatterplot specifically (rather than a plain two-column scatter) because the real business question isn't just 'are these two numbers related' — it's 'where do stroke cases cluster', so coloring by the target variable was essential.

Key Insight: Stroke cases (a distinct color) visibly cluster toward the older-age, higher-glucose region of the age vs. avg_glucose_level plot — a strong visual signal that age and glucose combined mark higher-risk patients.

8. Bivariate Analysis — Numeric vs Categorical (Stroke)

Do stroke patients differ in age, glucose, or BMI compared to non-stroke patients?

```
sns.boxplot(data=ok, x='stroke', y='age')
sns.boxplot(data=ok, x='stroke', y='avg_glucose_level')
sns.boxplot(data=ok, x='stroke', y='bmi')
sns.violinplot(data=ok, x='hypertension', y='age')
sns.violinplot(data=ok, x='heart_disease', y='age')
```

Why these variables: stroke was put on the x-axis in every boxplot because it's the target we're trying to explain — placing it on x lets us directly compare the y-value's box for stroke=0 vs stroke=1 side by side. hypertension and heart_disease vs age were added afterward because both conditions are known to become more common with age, so it's worth checking whether that pattern holds here too.

Key Insight: The age-vs-stroke boxplot shows the stroke=1 box sitting clearly higher than stroke=0 — stroke patients are older on average. Similarly, both the hypertension and heart_disease violin plots show wider, higher age distributions for patients who have the condition.

9. Bivariate Analysis — Categorical vs Categorical

How does each category (hypertension, smoking, gender, work type, etc.) relate to stroke?

```
sns.countplot(data=ok, x='hypertension', hue='stroke')
sns.countplot(data=ok, x='heart_disease', hue='stroke')
sns.countplot(data=ok, x='smoking_status', hue='stroke')
sns.countplot(data=ok, x='ever_married', hue='stroke')
sns.countplot(data=ok, x='gender', hue='stroke')
sns.countplot(data=ok, x='Residence_type', hue='stroke')
sns.countplot(data=ok, x='work_type', hue='stroke')
```

Why these variables: Every remaining categorical column was checked against stroke (hue='stroke') one at a time, rather than combining them, so each factor's individual relationship to stroke stays clearly readable in its own chart instead of becoming a cluttered multi-variable plot.

Key Insight: Notebook conclusion: 'Age, hypertension, heart disease, and high average glucose levels are the strongest factors associated with stroke risk. BMI and marital status have weaker or indirect relationships, while smoking remains an important modifiable risk factor.'

Multivariate Analysis

How do age, glucose, BMI, hypertension, heart disease, and work type interact together in relation to stroke?

```
sns.scatterplot(data=ok, x='age', y='avg_glucose_level', hue='stroke')
sns.scatterplot(data=ok, x='age', y='bmi', hue='stroke')
sns.scatterplot(data=ok, x='bmi', y='avg_glucose_level', hue='stroke')
```

```
sns.boxplot(data=ok, x='hypertension', y='age', hue='stroke')
sns.boxplot(data=ok, x='heart_disease', y='age', hue='stroke')
sns.boxplot(data=ok, x='smoking_status', y='age', hue='stroke')
sns.countplot(data=ok, x='hypertension', hue='heart_disease')
sns.boxplot(data=ok, x='work_type', y='age', hue='stroke')
```

Why these variables: This step repeats several earlier chart types but adds a third dimension (hue='stroke' ON TOP of an x/y that's already a category vs. numeric pairing, e.g. hypertension × age × stroke) specifically to check whether stroke risk changes WITHIN a subgroup — not just overall — which single bivariate charts can't show.

Key Insight: Notebook conclusion: 'Age remains the strongest factor influencing stroke risk, while high average glucose levels, hypertension, and heart disease further amplify that risk... Preventive strategies should focus on elderly individuals with chronic conditions.'

11. Group-Based Analysis with groupby()

What is the stroke breakdown within each category — gender, hypertension, smoking status, work type, etc.?

```
ok.groupby('gender')['stroke'].value_counts()
ok.groupby('hypertension')['stroke'].value_counts()
ok.groupby('heart_disease')['stroke'].value_counts()
ok.groupby('smoking_status')['stroke'].value_counts()
ok.groupby('work_type')['stroke'].value_counts()
ok.groupby('Residence_type')['stroke'].value_counts()
ok.groupby('ever_married')['stroke'].value_counts()
ok.groupby(['gender', 'smoking_status'])['stroke'].value_counts()
ok.groupby(['hypertension', 'heart_disease'])['stroke'].value_counts()
ok.groupby(['work_type', 'Residence_type'])['stroke'].value_counts()
```

Why these variables: groupby(...)['stroke'].value_counts() was used (instead of just mean()) because it shows the exact raw counts of stroke=0 vs stroke=1 within each group — useful for judging whether a group's sample size is even large enough to trust the pattern. The last three lines group by TWO columns at once specifically to check combined-risk combinations, like hypertension AND heart disease together.

Key Insight: hypertension=1 & heart_disease=1 patients had 13 strokes out of just 64 people (~20%) — the highest stroke rate of any combination — versus hypertension=0 & heart_disease=0, where only 149 out of 4,400 (~3.4%) had a stroke. This is the clearest evidence yet that combined conditions compound risk.

12. Pivot Table Analysis

Can I summarize average age, glucose, and BMI across groups in clean tables?

```
pd.pivot_table(ok, values='age', index='stroke', aggfunc='mean')
pd.pivot_table(ok, values='avg_glucose_level', index='hypertension', aggfunc='mean')
pd.pivot_table(ok, values='bmi', index='heart_disease', aggfunc='mean')
pd.pivot_table(ok, values='id', index='gender', columns='stroke', aggfunc='count')
pd.pivot_table(ok, values='id', index='smoking_status', columns='stroke',
aggfunc='count')
pd.pivot_table(ok, values='id', index='work_type', columns='stroke', aggfunc='count')
pd.pivot_table(ok, values='id', index=['hypertension', 'heart_disease'],
columns='stroke', aggfunc='count')
pd.pivot_table(ok, values='id', index='Residence_type', columns='gender',
aggfunc='count')
```

Why these variables: values='id' with aggfunc='count' was used specifically for the count-based pivots (gender, smoking, work_type) because id is unique per patient, so counting id rows is a safe way to count patients per group. values='age'/'avg_glucose_level'/'bmi' with aggfunc='mean' was used for the average-based pivots because we want a typical value per group, not a count.

Key Insight: Average Age by Stroke: 41.97 years (no stroke) vs. 67.73 years (stroke) — a 25+ year gap, the single strongest pattern found in Sprint 2. Average Glucose by Hypertension: 103.55 (no hypertension) vs. 130.19 (hypertension) — hypertensive patients run noticeably higher glucose too.

13. Crosstab Analysis

How do two categorical columns overlap — e.g. gender vs smoking status, or hypertension vs heart disease?

```
pd.crosstab(ok['gender'], ok['smoking_status'])
pd.crosstab(ok['work_type'], ok['Residence_type'])
pd.crosstab(ok.ever_married, ok.work_type)
pd.crosstab(ok.hypertension, ok.heart_disease)
pd.crosstab(ok['gender'], ok['ever_married'], margins=True)
pd.crosstab(ok['smoking_status'], ok['work_type'])
pd.crosstab(ok['Residence_type'], ok['ever_married'])
pd.crosstab(ok['gender'], ok['work_type'])
```

Why these variables: margins=True was added only to the gender × ever_married crosstab specifically to get row/column totals for a sanity check (5,110 total). The other crosstabs skip margins because their purpose is purely to inspect the relationship shape between two categories, not to re-verify total row counts again.

Key Insight: hypertension × heart_disease crosstab shows only 64 people have both conditions, out of 5,110 — a small but, per Task 11, disproportionately high-risk group. The gender × ever_married crosstab confirms totals add up correctly to 5,110, validating the earlier group-based counts.

14. Correlation Analysis

Which numeric features move together the most, and which correlate most with stroke?

```
corr = ok[['age', 'avg_glucose_level', 'bmi', 'hypertension', 'heart_disease',
'stroke']].corr()
print(corr.round(2))
sns.heatmap(corr)
plt.show()
corr['stroke'].sort_values(ascending=False)
```

Why these variables: Only these six specific columns were included in corr() (rather than all numeric columns) because id has no real-world meaning to correlate with anything — including it would just add meaningless noise to the heatmap. corr['stroke'].sort_values() was added specifically to rank every feature's relationship to the target variable from strongest to weakest.

Key Insight: Ranked correlation with stroke: age (0.25) > heart_disease (0.13) ≈ avg_glucose_level (0.13) ≈ hypertension (0.13) > bmi (0.04). Age is clearly the single strongest linear predictor of stroke among these numeric features, while BMI's direct correlation is surprisingly weak.

15. Full Visualization Gallery

What's the best chart type for each different kind of finding?

```
sns.histplot(ok['age'], kde=True)
sns.countplot(data=ok, x='stroke')
sns.boxplot(data=ok, x='stroke', y='avg_glucose_level')
sns.violinplot(data=ok, x='stroke', y='age')
sns.scatterplot(data=ok, x='age', y='avg_glucose_level', hue='stroke')
ok.groupby('work_type')['stroke'].mean().plot(kind='bar')
sns.pairplot(ok, vars=['age', 'bmi', 'avg_glucose_level'], hue='stroke')
sns.heatmap(ok.corr(numeric_only=True), annot=True)
```

Why these variables: groupby('work_type')['stroke'].mean().plot(kind='bar') specifically plots the stroke RATE (a proportion, 0–1) per work type rather than raw counts — this matters because work types have very

different total sizes (e.g. only 22 Never_Worked vs. 2,925 Private), so a raw-count bar chart would be misleading. `sns.pairplot(vars=[...])` was limited to just 3 numeric columns to keep the resulting grid of charts readable.

Key Insight: `countplot(x='stroke')` visually confirms severe class imbalance: the `stroke=0` bar completely dwarfs the `stroke=1` bar. This single chart is arguably the most important one in the whole EDA — any future model built on this data must account for this imbalance.

16. Extra Check — Normality Test on Glucose Level

Is average glucose level normally distributed, or skewed?

```
from statsmodels.stats.stattools import jarque_bera
jb_stat, p_value, skew, kurtosis = jarque_bera(df['avg_glucose_level'])
print("JB:", jb_stat)
print("p_value:", p_value)
print("skewness:", skew)
print("Kurtosis:", kurtosis)
```

Why these variables: The Jarque-Bera test was used specifically (rather than just eyeballing the histogram) because it gives an exact numeric p-value for whether a column is normally distributed — this decides whether a simple t-test (needs roughly-normal data) or a non-parametric test like Wilcoxon would be more appropriate for glucose level.

Key Insight: JB statistic = 2703.4, p-value = 0.0 (far below 0.05), skewness = 1.57 (strongly right-skewed). This confirms mathematically what the histogram suggested: `avg_glucose_level` is NOT normally distributed — it has a long right tail of high-glucose patients.

Sprint 2 Business Insights : Stroke occurrence is highly imbalanced (far fewer stroke than non-stroke cases). Older patients have a much higher stroke risk (67.7 vs. 42.0 years average). Elevated glucose, hypertension, and heart disease are all strongly linked to stroke. BMI has only a weak direct relationship. Smoking shows varying stroke rates but many records are 'Unknown'. Urban vs. rural residence shows little difference. Recommended: screen patients 50+, monitor hypertension/heart disease patients closely, promote glucose testing, and improve smoking-status data collection.

Sprint 3 – Advanced Statistics & Feature Engineering

1. Central Tendency — Age, Glucose, BMI

What are the exact mean, median, and mode of the three key numeric features?

```
print(f"Age - Mean: {ok['age'].mean():.2f}")
print(f"Age - Median: {ok['age'].median():.2f}")
print(f"Age - Mode: {ok['age'].mode()[0]:.2f}")

print(f"Average Glucose Level - Mean: {ok['avg_glucose_level'].mean():.2f}")
print(f"Average Glucose Level - Median: {ok['avg_glucose_level'].median():.2f}")
print(f"Average Glucose Level - Mode: {ok['avg_glucose_level'].mode()[0]:.2f}")

print(f"BMI - Mean: {ok['bmi'].mean():.2f}")
print(f"BMI - Median: {ok['bmi'].median():.2f}")
print(f"BMI - Mode: {ok['bmi'].mode()[0]:.2f}")
```

Why these variables: mode()[0] specifically takes the FIRST most-frequent value, because a column can technically have more than one mode (a tie) — indexing [0] guarantees the code always returns one clean number instead of crashing or printing a list.

Key Insight: Age: mean 43.23, median 45.00, mode 78.00. Glucose: mean 106.15, median 91.88, mode 93.88. BMI: mean 28.86, median 28.10, mode 28.10 — BMI's three measures sit close together (fairly balanced distribution), while glucose's mean is well above its median (confirming the right-skew already found).

2. Dispersion — Variance, Std Dev, IQR

How much does each of these three features vary from patient to patient?

```
print(f"Age - Variance: {ok['age'].var():.2f}")
print(f"Age - Standard Deviation: {ok['age'].std():.2f}")
print(f"Age - IQR: {ok['age'].quantile(0.75) - ok['age'].quantile(0.25):.2f}")

print(f"Average Glucose Level - Variance: {ok['avg_glucose_level'].var():.2f}")
print(f"Average Glucose Level - Standard Deviation: {ok['avg_glucose_level'].std():.2f}")
print(f"Average Glucose Level - IQR: {ok['avg_glucose_level'].quantile(0.75) - ok['avg_glucose_level'].quantile(0.25):.2f}")

print(f"BMI - Variance: {ok['bmi'].var():.2f}")
print(f"BMI - Standard Deviation: {ok['bmi'].std():.2f}")
print(f"BMI - IQR: {ok['bmi'].quantile(0.75) - ok['bmi'].quantile(0.25):.2f}")
```

Why these variables: IQR was computed manually as quantile(0.75) minus quantile(0.25) (rather than using a separate library function) because it's a one-line calculation and keeps this cell self-contained and easy to read without an extra import.

Key Insight: Age: variance 511.33, std 22.61, IQR 36.00 — a wide, evenly-spread range. Glucose: variance 2,050.60, std 45.28, IQR 36.84 — the largest spread of the three by far, again confirming glucose is the most variable and skewed feature.

3. Distribution Analysis — Side-by-Side Histograms

Can I compare the shape of age, glucose, and BMI's distributions at a glance?

```
plt.figure(figsize=(18, 5))
plt.subplot(1, 3, 1); sns.histplot(ok['age'], kde=True); plt.title('Distribution of Age')
```

```
plt.subplot(1, 3, 2); sns.histplot(ok['avg_glucose_level'], kde=True);
plt.title('Distribution of Average Glucose Level')
plt.subplot(1, 3, 3); sns.histplot(ok['bmi'], kde=True); plt.title('Distribution of BMI')
plt.tight_layout(); plt.show()
```

Why these variables: `plt.subplot(1, 3, ...)` was used to place all three histograms in a single row of one figure (rather than three separate `plt.show()` calls) specifically so the three shapes can be visually compared side by side in one glance, instead of scrolling between three different outputs.

Key Insight: The three panels visually confirm: age looks broad and only mildly skewed, `avg_glucose_level` shows a clear long right tail (skewed), and `bmi` looks close to a normal bell shape with a slight right skew — matching the numeric skew/kurtosis values already calculated.

4. Correlation and Covariance Matrices

Which numeric features are related, and in what direction/units?

```
numerical_cols = ok.select_dtypes(include=['float64', 'int64']).columns
correlation_matrix = ok[numerical_cols].corr()
display(correlation_matrix)
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.show()

covariance_matrix = ok[numerical_cols].cov()
display(covariance_matrix)
```

Why these variables: `select_dtypes(include=['float64', 'int64'])` was used here (rather than the earlier hand-picked 6-column list) because this Sprint 3 step wants the FULL numeric picture including `id`, to double check nothing meaningful was missed — `id`'s near-zero correlation with everything then confirms it was safe to exclude from Sprint 2's more focused analysis.

Key Insight: Correlation with stroke, ranked: age (0.245) > heart_disease (0.135) > hypertension (0.128) ≈ `avg_glucose_level` (0.132) > `bmi` (0.036). The covariance matrix shows the same direction of relationships but in raw (unstandardized) units — correlation is more interpretable, which is why it's used for the ranking conclusions.

5. Hypothesis Test #1 — T-Test: Glucose Level vs Stroke

Is the glucose-level difference between stroke and non-stroke patients real, or could it be random chance?

```
from scipy import stats

glucose_stroke = ok[ok['stroke'] == 1]['avg_glucose_level'].dropna()
glucose_no_stroke = ok[ok['stroke'] == 0]['avg_glucose_level'].dropna()

t_stat, p_value = stats.ttest_ind(glucose_stroke, glucose_no_stroke)
print(f"Test Statistic (t-value): {t_stat:.2f}")
print(f"P-value: {p_value}")

alpha = 0.05
if p_value < alpha:
    decision = "Reject the Null Hypothesis"
else:
    decision = "Fail to reject the Null Hypothesis"
print(f"Decision: {decision}")
```

Why these variables: A T-test was chosen specifically (over Chi-square or ANOVA) because we're comparing the MEAN of one numeric variable (glucose) between exactly TWO groups (`stroke=1` vs `stroke=0`) — that's

the exact scenario a two-sample T-test is designed for. $\alpha=0.05$ is the standard significance threshold used across all three hypothesis tests here, for consistency.

Key Insight: H_0 : no real difference in glucose levels between stroke/non-stroke groups. Result: $t=9.51$, $p=2.77 \times 10^{-21}$ (essentially 0, far below 0.05) → Reject H_0 . Business interpretation: the higher glucose levels seen in stroke patients are a real, statistically significant pattern, not random noise.

6. Hypothesis Test #2 — Chi-Square: Hypertension vs Stroke

Is hypertension genuinely associated with stroke, or could that pattern be coincidence?

```
contingency_table = pd.crosstab(ok['hypertension'], ok['stroke'])
chi2_stat, p_value, dof, expected_freq = stats.chi2_contingency(contingency_table)
print(f"Chi-Square Statistic: {chi2_stat:.2f}")
print(f"P-value: {p_value:.3f}")

alpha = 0.05
if p_value < alpha:
    decision = "Reject the Null Hypothesis"
else:
    decision = "Fail to reject the Null Hypothesis"
print(f"Decision: {decision}")
```

Why these variables: A Chi-Square test was chosen (instead of a T-test) because both hypertension and stroke are CATEGORICAL (0/1) variables — Chi-square is specifically designed to test whether two categorical variables are independent, using a contingency table (crosstab) rather than group means.

Key Insight: H_0 : hypertension and stroke are unrelated. Result: $\chi^2=81.61$, $p \approx 0.000$ (far below 0.05) → Reject H_0 . Business interpretation: there IS a statistically significant association between hypertension and stroke occurrence — this is not a coincidental pattern.

7. Hypothesis Test #3 — ANOVA: Glucose Level Across Work Types

Does average glucose level genuinely differ across the 5 work-type groups?

```
from scipy.stats import f_oneway

work_types = df['work_type'].unique()
groups = [ok['avg_glucose_level'][ok['work_type'] == wt].dropna() for wt in work_types]

f_stat, p_value = f_oneway(*groups)
print(f"Test Statistic (F-value): {f_stat:.2f}")
print(f"P-value: {p_value:.3f}")

alpha = 0.05
if p_value < alpha:
    decision = "Reject the Null Hypothesis"
else:
    decision = "Fail to reject the Null Hypothesis"
print(f"Decision: {decision}")
```

Why these variables: ANOVA (not a T-test) was chosen here specifically because there are FIVE work-type groups to compare at once (Private, Self-Employed, Govt_Job, Children, Never_Worked) — a T-test only handles two groups, while one-way ANOVA is built to compare the means of three or more groups in a single test.

Key Insight: H_0 : glucose level is the same across all work types. Result: $F=16.61$, $p \approx 0.000$ (far below 0.05) → Reject H_0 . Business interpretation: average glucose level genuinely differs by work type, suggesting

occupational or lifestyle factors worth investigating further (though this doesn't mean occupation directly causes it).

8. Confidence Intervals for Age, Glucose, and BMI

What range can we be 95% confident the TRUE population average falls within, for each key feature?

```
import numpy as np
from scipy.stats import t

# --- Age ---
age_mean = ok['age'].mean(); age_std = ok['age'].std(); age_n = len(ok['age'])
t_score = t.ppf(1 - 0.05/2, age_n - 1)
margin_of_error_age = t_score * (age_std / np.sqrt(age_n))
confidence_interval_age = (age_mean - margin_of_error_age, age_mean +
margin_of_error_age)
print(confidence_interval_age)

# --- Average Glucose Level ---
glucose_mean = ok['avg_glucose_level'].mean(); glucose_std =
ok['avg_glucose_level'].std(); glucose_n = len(ok['avg_glucose_level'])
t_score = t.ppf(1 - 0.05/2, glucose_n - 1)
margin_of_error_glucose = t_score * (glucose_std / np.sqrt(glucose_n))
confidence_interval_glucose = (glucose_mean - margin_of_error_glucose, glucose_mean +
margin_of_error_glucose)
print(confidence_interval_glucose)

# --- BMI ---
bmi_series = ok['bmi'].dropna()
bmi_mean = bmi_series.mean(); bmi_std = bmi_series.std(); bmi_n = len(bmi_series)
t_score = t.ppf(1 - 0.05/2, bmi_n - 1)
margin_of_error_bmi = t_score * (bmi_std / np.sqrt(bmi_n))
confidence_interval_bmi = (bmi_mean - margin_of_error_bmi, bmi_mean +
margin_of_error_bmi)
print(confidence_interval_bmi)
```

Why these variables: t.ppf() (the t-distribution) was used instead of a plain normal z-score because it's the statistically correct choice whenever we estimate a confidence interval from SAMPLE standard deviation rather than a known population standard deviation — which is exactly our situation with all three features. bmi_series.dropna() was added only for BMI specifically since — even after Sprint 1's median fill — this recalculation intentionally works from the distribution's own spread.

Key Insight: 95% CI for Age: (42.61, 43.85) years. 95% CI for Average Glucose Level: (104.91, 107.39). 95% CI for BMI: (28.65, 29.07). In plain terms: we're 95% confident the TRUE average age of all patients like these is between about 42.6 and 43.9 years — a tight, reliable range given the large sample size (5,110).

9. Feature Engineering #1 — Age Group

Can I turn the continuous 'age' column into meaningful groups (Children/Adolescents/Adults/Seniors)?

```
age_bins = [0, 12, 19, 64, ok['age'].max()]
age_labels = ['Children', 'Adolescents', 'Adults', 'Seniors']
ok['Age_Group'] = pd.cut(df['age'], bins=age_bins, labels=age_labels, right=False)
display(ok['Age_Group'].value_counts())
```

Why these variables: The bin edges [0, 12, 19, 64, max] specifically mirror standard age-group definitions used in public health (childhood, adolescence, working-age adults, seniors) rather than arbitrary round numbers — this makes the resulting groups clinically meaningful, not just a mathematical split. right=False means each bin includes its left edge (e.g. 65 falls into Seniors, not Adults).

Key Insight: Group sizes: Adults 3,114, Seniors 1,024, Children 543, Adolescents 373. Most patients in this dataset are working-age adults, but the Seniors group — though smaller — turns out (see Task 11) to carry by far the highest stroke rate.

10. Feature Engineering #2 — BMI Category

Can I turn the numeric 'bmi' column into standard medical categories?

```
bmi_bins = [0, 18.5, 24.9, 29.9, ok['bmi'].max()]
bmi_labels = ['Underweight', 'Normal Weight', 'Overweight', 'Obese']
ok['BMI_Category'] = pd.cut(df['bmi'], bins=bmi_bins, labels=bmi_labels, right=True,
include_lowest=True)
display(ok['BMI_Category'].value_counts())
```

Why these variables: The bin edges (18.5 / 24.9 / 29.9) are not arbitrary — they are the standard WHO/medical BMI classification thresholds, chosen specifically so the new column means something to a doctor immediately, rather than an invented split. include_lowest=True ensures a BMI of exactly 0 (the lower bound) would still be correctly classified rather than dropped as NaN.

Key Insight: Group sizes: Obese 1,920, Overweight 1,610, Normal Weight 1,231, Underweight 349. More than two-thirds of patients (68.9%) fall into Overweight or Obese — a notable population-health finding on its own, before even connecting it to stroke.

11. Feature Engineering #3 — Composite Risk Category

Can I combine hypertension, heart disease, high glucose, and high BMI into one overall risk label?

```
high_glucose_threshold = 125
ok['High_Glucose'] = (ok['avg_glucose_level'] >= high_glucose_threshold).astype(int)
ok['High_BMI'] = (ok['BMI_Category'] == 'Obese').astype(int)

ok['Risk_Score'] = ok['hypertension'] + ok['heart_disease'] + ok['High_Glucose'] +
ok['High_BMI']

def get_risk_category(score):
    if score <= 1:
        return 'Low Risk'
    elif score == 2:
        return 'Medium Risk'
    else:
        return 'High Risk'

ok['Risk_Category'] = ok['Risk_Score'].apply(get_risk_category)
display(ok['Risk_Category'].value_counts())
```

Why these variables: The threshold 125 mg/dL for High_Glucose was chosen specifically because it is a recognized clinical cutoff associated with diabetes risk, not a round number picked at random. Risk_Score simply adds up 4 binary flags (0 or 1 each), so it naturally ranges 0-4 — the get_risk_category() cutoffs (≤ 1 / $= 2$ / ≥ 3) were chosen to split that 0-4 range into three roughly meaningful tiers rather than four overly-thin categories.

Key Insight: Distribution: Low Risk 4,285, Medium Risk 634, High Risk 191. Even though High Risk is the smallest group, it's exactly the group we'd want doctors to focus limited screening resources on first — the next task confirms this group does have the highest stroke rate.

12. Feature Evaluation — Do the New Features Actually Work?

Do Age_Group, BMI_Category, and Risk_Category actually line up with real stroke rates?

```
stroke_by_age_group =
ok.groupby('Age_Group')['stroke'].mean().sort_values(ascending=False)
display(stroke_by_age_group)

stroke_by_bmi_category =
ok.groupby('BMI_Category')['stroke'].mean().sort_values(ascending=False)
display(stroke_by_bmi_category)

stroke_by_risk_category =
ok.groupby('Risk_Category')['stroke'].mean().sort_values(ascending=False)
display(stroke_by_risk_category)
```

Why these variables: .mean() on the 0/1 stroke column (grouped by each new feature) was used specifically because averaging a 0/1 column directly gives the PROPORTION of stroke cases in that group — a clean, comparable stroke rate — rather than a raw count that would be biased by group size. sort_values(ascending=False) ranks each feature's categories from highest to lowest risk, making the pattern immediately visible.

Key Insight: Age_Group: Seniors 14.94% stroke rate vs. Adults 2.73%, Adolescents 0.27%, Children 0.18% — a dramatic, clean gradient. BMI_Category: Overweight 7.14% and Obese 5.10% vs. Normal Weight 2.84% and Underweight 0.29%. Risk_Category: High Risk 17.28% vs. Medium Risk 10.09% vs. Low Risk 3.55% — proving the composite Risk_Category is nearly 5× more predictive at its top tier than at its bottom tier, confirming all three engineered features add genuine value.

Sprint 3 Key Findings: 1) Age is the primary risk factor — stroke incidence rises sharply with age, peaking in Seniors. 2) The T-test confirms glucose level is genuinely higher in stroke patients ($p \approx 0$). 3) The Chi-square test confirms hypertension is genuinely associated with stroke ($p \approx 0$). 4) The ANOVA test confirms glucose level genuinely varies by work type ($p \approx 0$). 5) Obese/Overweight patients show a higher stroke proportion than Normal/Underweight. 6) The engineered Risk_Category cleanly stratifies patients into Low/Medium/High risk, with High Risk patients nearly 5× more likely to have had a stroke than Low Risk patients.

Final Business Recommendations

1. Targeted screening for seniors (65+), especially with other comorbidities.
2. Early detection and management of elevated glucose levels through regular monitoring.
3. Aggressive hypertension control given its strong statistical link to stroke.
4. Investigate occupational health factors behind the work-type glucose differences.
5. Promote healthy-weight programs given the Overweight/Obese stroke gap.
6. Use the Risk_Category framework in clinical decision-making — prioritize 'High Risk' patients for proactive follow-up.
7. Adopt a holistic view of patient risk — combine age, glucose, hypertension, and heart disease rather than looking at any one factor alone.